

Why Simple RAG Wins

The design behind this pipeline: one retrieval, one model call, answers cited to the page. Chosen for latency, cost, and scale.

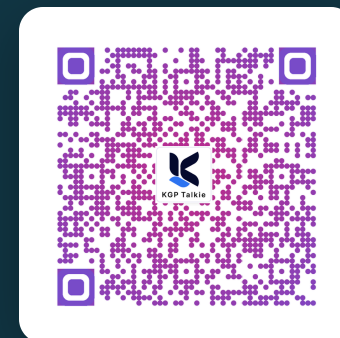
page chunks

hybrid search

metadata filters

grounded prompt

1 model call



 **KGP Talkie**
LEARN. BUILD. TALK AI

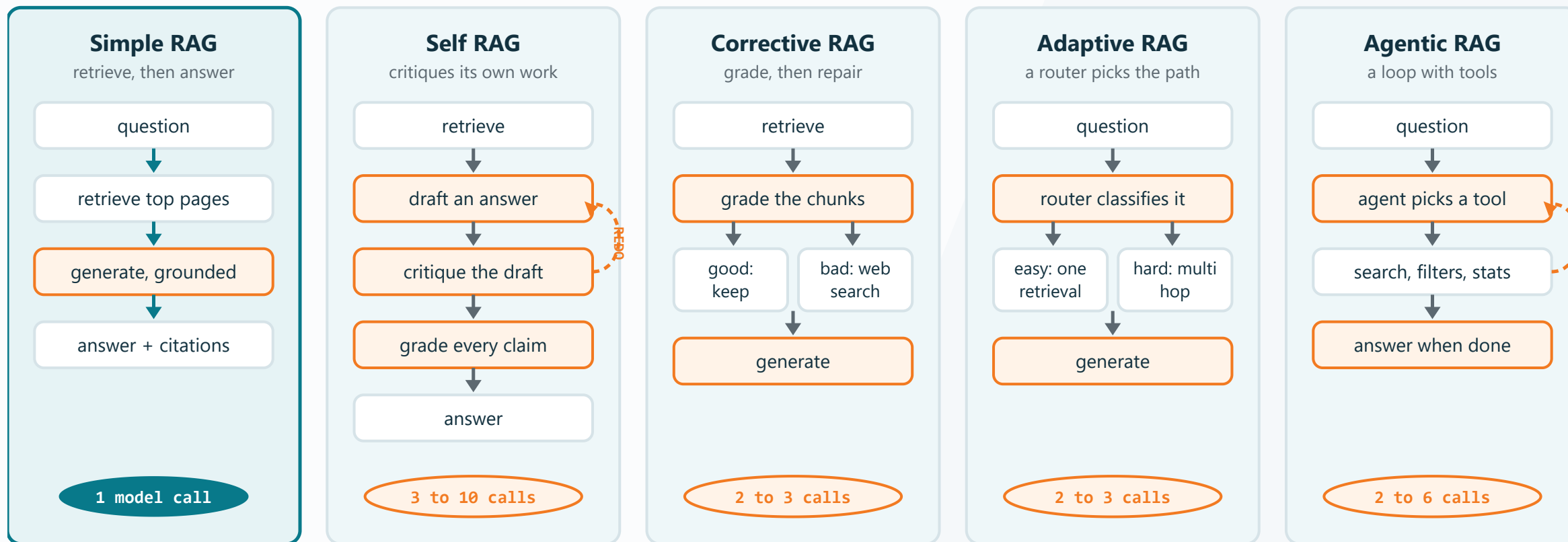
Scan to subscribe

The RAG Family, in One Look

THIS PIPELINE

 one LLM call no model involved

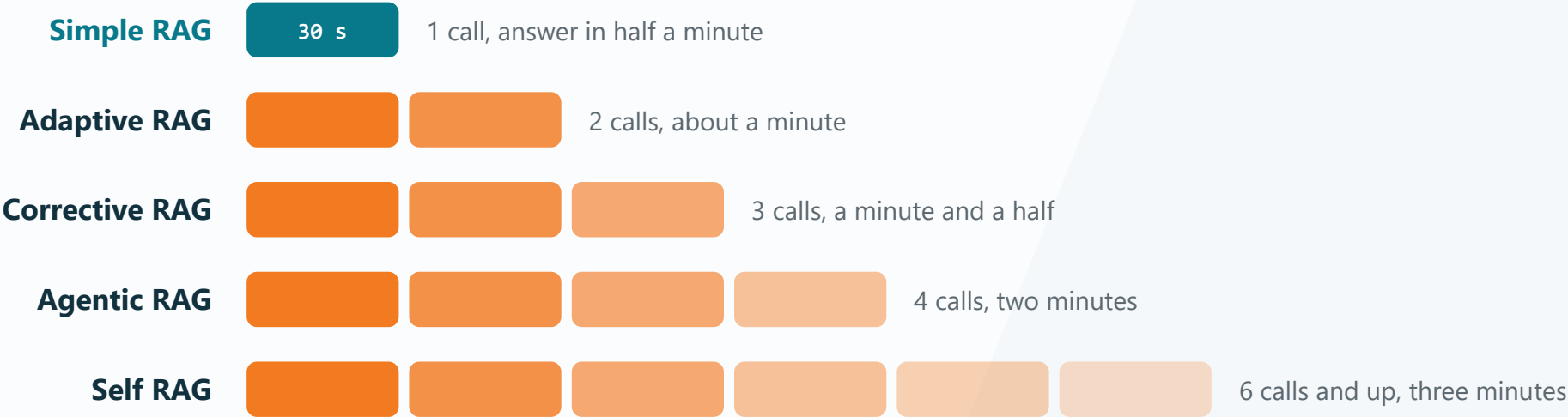
count the orange boxes on each path



Every variant exists to fix a retrieval mistake **after** it happens. Each fix is **another model call** on every question, answered well or not.

What Extra Calls Cost

MODEL CALLS PER QUESTION • ONE CALL RUNS ABOUT 30 SECONDS ON A LOCAL 27B



LATENCY

Users feel every call

Half a minute survives a chat UI. Three minutes does not, and the critique passes run on every question.

COST

Tokens are billed per call

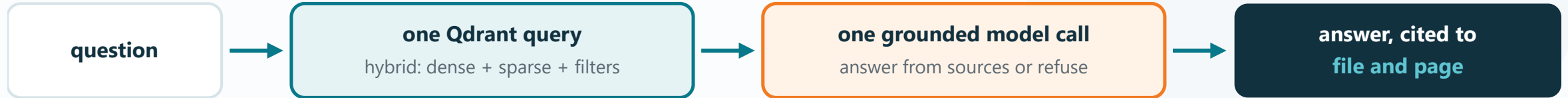
A critique pass re-reads the same retrieved context again. Six calls means paying for the context six times.

SCALABILITY

A GPU serves so many calls

Six calls per question serves one sixth of the users on the same hardware. Extra calls are divided out of throughput.

Why This Simple Design Works



PAGE CHUNKS

One chunk is one page, so a citation is a page a reader can open and check.

HYBRID + FILTERS

Sparse catches AWS and 10-K, dense catches meaning, filters narrow. Zero model calls.

GROUNDING PROMPT

Rule 3 refuses when sources fall short. The guardrail rides the one call, free of charge.

LEFT OUT ON PURPOSE

retrieval grader

+1 call per question

self critique loop

+2 to 5 calls

query router

+1 call per question

reranker model

+1 model to host

347 pages · 3 filings

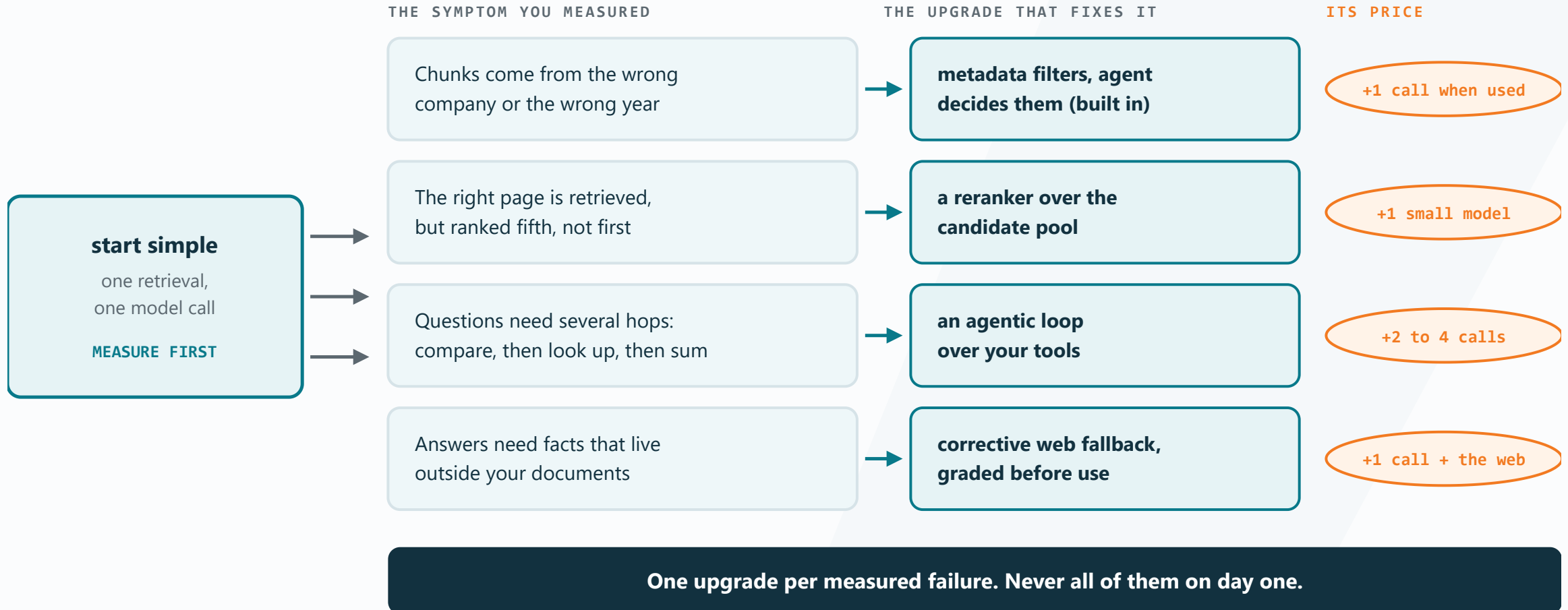
~30 s warm answer

\$0 per query, fully local

one laptop runs all of it

Good pages in, grounded answer out: retrieval is fixed **before** generation, so nothing needs correcting after. When a question truly needs it, the agent adds **one** planning call, and only then.

When the Complex Ones **Earn** Their Calls



This pipeline already took exactly one step on that ladder: the **agent decides filters**, because wrong-company chunks were a failure you could **see** in section 8.